

# Esame OS Internals (Cabodi)

Data: 20/06/2022

---

*Nota Generale: TUTTE LE RISPOSTE SI/NO DEVONO ESSERE MOTIVATE. QUANDO I RISULTATI SONO NUMERI, SONO NECESSARI IL RISULTATO FINALE E I RELATIVI PASSI INTERMEDI (O FORMULE).*

## 1 SECTION 1: THEORETICAL (Silberschatz)

### Domanda 1 (Memoria Virtuale, TLB e EAT)

**Contesto:** Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di TLB (Translation Look-aside Buffer). La tabella delle pagine ("page-table") viene realizzata con uno schema a due livelli, nel quale un indirizzo logico di 64 bit viene suddiviso (da MSB a LSB) in 3 parti:  $p_1$ ,  $p_2$  e  $d$ ,  $p_2$  ha 14 bit e  $d$  13 bit. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. La memoria virtuale viene gestita con paginazione a richiesta.

**Si risponda alle seguenti domande:**

- A) Supponendo che la memoria RAM abbia tempo di accesso di 75 ns, si calcoli il TLB miss ratio (frequenza di miss), sapendo che, a causa dei TLB miss, si sa che  $EAT_{PT} \geq 100$  ns, assumendo che il tempo di accesso alla TLB sia trascurabile. Si tratta di un lower bound o di un upper bound per la TLB miss ratio?
- A) Si consideri ora la frequenza di page fault  $P_{PF}$ . Una valutazione sperimentale mostra che un page fault viene servito in 2 ms in media, e che il degrado di prestazioni (aumento di tempo di esecuzione) causato dai page fault è compreso tra 10% e 30%. Calcolare l'intervallo di valori di  $P_{PF}$  compatibile con la valutazione sperimentale, assumendo  $EAT_{PT} = 100$  ns.
- A) Una seconda valutazione sperimentale (una simulazione) viene fatta usando due stringhe di riferimento  $w_1$  e  $w_2$  aventi lunghezza, rispettivamente,  $len(w_1) = 10^6$ ,  $len(w_2) = 2 \cdot 10^6$  e probabilità di stringa  $p_1$  e  $p_2$ . Le simulazioni hanno generato 400 page fault in totale (100 su  $w_1$  e 300 su  $w_2$ ) e stimato una probabilità empirica  $f = 0.00012$  (frequenza attesa) che avvenga un page fault nel sistema reale. Si calcolino i valori di  $p_1$  e  $p_2$ .

---

### Domanda 2 (File System Unix-like e Inodes)

**Contesto:** Considerare un file system Unix-like, basato su inode, con 13 puntatori / indici (10 diretti, 1 singolo indiretto, 1 doppio indiretto e 1 triplo indiretto). I puntatori / indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di 2 KB. Il file system risiede su una partizione del disco di 800 GB, che include sia blocchi di dato che blocchi di indice.

**Si risponda alle seguenti domande:**

- A) Supponendo che tutti i metadati (eccetto i blocchi indice) abbiano dimensioni trascurabili, calcolare il numero massimo di file che il file system può ospitare, utilizzando l'indicizzazione indiretta doppia (N2) e l'indicizzazione indiretta tripla (N3).
- A) Dato un file binario di dimensione 10240.5 KB, calcolare esattamente quanti blocchi indice e blocchi di dato occupa il file.
- A) Considerare lo stesso file della domanda B, su cui viene chiamata l'operazione `lseek (fd, offset, SEEK_END)` per posizionare l'offset del file per la successiva operazione di lettura / scrittura (`SEEK_END` significa che l'offset viene calcolato dalla fine del file). Supponiamo che `fd` sia il descrittore di file associato al file (già aperto), e `offset = 0x00800000`.
1. Calcolare il numero di blocco logico (numerazione relativa ai blocchi del file, numerati a partire da 0) in corrispondenza del quale viene spostata la posizione (da `lseek`).
  2. Se il file utilizza un'indicizzazione indiretta singola (o doppia / tripla), calcolare quale riga del blocco di indici esterno contiene l'indice del blocco di dati (o l'indice del blocco di indici più interno).
- 

### Domanda 3 (Dispositivi di I/O, Polling e Interrupts)

**Contesto:** Un processo user vuole effettuare un input da un dispositivo di IO a caratteri (character device) mediante una strategia basata su polling del dispositivo. Per evitare un'attesa troppo lunga (nel loop di polling) da parte del driver del dispositivo, si chiede all'autore del programma eseguito dal processo user, di fare direttamente polling mediante un loop di lettura del registro di stato del dispositivo.

**Si risponda alle seguenti domande:**

- A) E' possibile effettuare questa operazione? (se sì, dire come, se no, dire perché).
- A) Supponendo che il dispositivo sia associato alla tastiera e lo si gestisca in interrupt, ci si aspetta che venga generato un interrupt per ogni carattere ricevuto oppure un interrupt per ogni "invio" (cioè enter, fine-riga)? (Motivare).
- A) Dato un altro dispositivo di IO, gestito a blocchi: E' possibile che una sola scrittura (mediante chiamata a `write()`) tenti di fare output di più di una coppia puntatore/dimensione?
- A) Dato il driver che realizza la scrittura, supponendo che il sistema sia dotato di DMA controller, è possibile che venga eseguita "prima" una operazione di lettura e/o scrittura attivata successivamente a questa, da un altro processo? E dallo stesso processo? (Motivare).

## 2 SECTION 2: OS161

### Domanda 4 (Sincronizzazione Produttore/Consumatore)

**Contesto:** Si considerino tre kernel thread in OS161, i quali implementano un'attività di trasferimento dati basata sul modello produttore/consumatore (due produttori, più consumatori). I thread condividono una struttura C di tipo `struct prodCons` definita come segue:

```
#define NumP 2
struct prodCons {
    void *data[NumP];
    int size[NumP];
    int busy[NumP];
    int ready[NumP];
    struct lock *pc_lk;
    struct cv *pc_cv;
    /* other stuff omitted */
};
```

Il produttore  $i$  (con  $i = 0$  oppure 1) aggiorna le  $i$ -esime entry nei vettori `data` e `size` (dato e dimensione, in pratica è un buffer). I consumatori possono leggere indifferentemente una delle due caselle dei vettori. Il lock viene utilizzato per la mutua esclusione sui vettori condivisi. I vettori di flag `ready` e `busy` indicano, se valgono 1:

- `ready`: dato (e dimensione) corrispondente pronto per essere letto
- `busy`: dato attualmente in lettura o in scrittura

Prima di lavorare sui dati, un consumatore deve quindi attendere/verificare che una opportuna condizione sui flag sia vera. Questo avviene chiamando la funzione: `int consumerWait(struct prodCons *pc);`

**Si risponda alle seguenti domande:**

- B) La struttura condivisa può essere posizionata nello stack di thread o dovrebbe essere una variabile globale o altro?
- B) Dato che i campi `pc_lk` e `pc_cv` sono puntatori, dove dovrebbero essere chiamate `lock_create()` e `cv_create()`? Nei thread produttori, nel thread consumatore? Altrove?
- B) Si fornisca un'implementazione della funzione `consumerWait`. Non è necessario alcun codice produttore e/o consumatore, solo la funzione, tenendo presente che la funzione ha come scopo ritornare l'indice (0 o 1) del buffer da cui leggere (avendone posto a 1 il corrispondente flag `busy`/letto). Il dato non va letto nella funzione.

---

### Domanda 5 (Tabelle File di Processo e di Sistema)

**Contesto:** Si consideri in OS161 una possibile implementazione delle system call relative al file system, basate su una tabella di processo e su una di sistema, definite come segue:

```
/* system open file table */
struct openfile {
    struct vnode *vn;
    off_t offset;
    unsigned int countRef;
```

```
};

/* this is a global variable */
struct openfile systemFileTable[SYSTEM_OPEN_MAX];

/* user open file table: this is a field of struct proc */
struct openfile *fileTable[OPEN_MAX];
```

**Si risponda alle domande seguenti (motivando le risposte):**

- B)** Perché `systemFileTable` è una variabile globale, mentre `fileTable` è un campo di `struct proc`?
- B)** Si supponga che, dati due processi user P1 e P2 (con `p1` e `p2` puntatori alle rispettive `struct proc`) siano vere le seguenti uguaglianze:
- `p1->fileTable[5] == &systemFileTable[20]`
  - `p2->fileTable[8] == &systemFileTable[20]`

A quale/i file descriptor (fd) corrisponde/corrispondono le parti di tabella visualizzate? 5, 8, 20? (Motivare). Se P1 e P2 fanno solo letture sul/sui file di cui sopra, le letture sono indipendenti oppure c'è interazione tra i processi?

- B)** Si supponga di aver implementato il supporto per la system call `SYS_lseek`. Una chiamata a `lseek` effettuata da P1 (si veda domanda B) influenzerà la successiva `read` effettuata da P2:
1. sempre?
  2. mai?
  3. solo se il parametro offset è `SEEK_CUR`?

(sono possibili selezioni multiple, per ogni selezione occorre motivare la risposta).